

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0716

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate segment (length m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csmalog.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

1. Star Topology Generator

Approach:

- Get the number of hosts from the user.
- Create one central switch.
- Assign each host a Cat6 UTP cable length.
- Display every host → switch connection.
- Verify that all hosts are connected

```
main.py Visualize Run Output
1 def generate_star_topology():
2     try:
3         n = int(input("Enter number of hosts: "))
4
5         if n <= 0:
6             print("Error: Number of hosts must be greater
7                 than 0.")
8             return
9
10        switch_name = "Central-Switch"
11        connections = []
12
13        print("\n===== STAR TOPOLOGY =====")
14        print(f"                [{switch_name}]")
15
16        for i in range(1, n + 1):
17            host = f"Host-{i}"
18
19            length = float(input(
20                f"Enter Cat6 cable length for {host} (meters)
21                : "
22            ))
23
24            if length <= 0 or length > 100:
25                print("Invalid cable length. Must be between
26                    0 and 100 meters.")
27                return
```

Enter number of hosts: 4

===== STAR TOPOLOGY =====

[Central-Switch]

Enter Cat6 cable length for Host-1 (meters): 10

Enter Cat6 cable length for Host-2 (meters): 15

Enter Cat6 cable length for Host-3 (meters): 20

Enter Cat6 cable length for Host-4 (meters): 12

Topology Connections:

Host-1	----	10.0 m	---->	Central-Switch
Host-2	----	15.0 m	---->	Central-Switch
Host-3	----	20.0 m	---->	Central-Switch
Host-4	----	12.0 m	---->	Central-Switch

Verification: SUCCESS

All 4 hosts are connected through Central-Switch.

=== Code Execution Successful ===

Explanation

In a **star topology**, every host has a separate connection to a central switch. If one cable fails, only that particular host is affected. The remaining hosts can continue communicating through the switch. This makes star topology reliable, easy to troubleshoot, and commonly used in LANs.

2. UTP Cable Length Validation

Approach:

The IEEE 802.3 Ethernet standard specifies a maximum copper Ethernet cable segment length of **100 metres** for the common 100BASE-T/1000BASE-T structured cabling channel model.

The function therefore accepts lengths from 0 to 100 m.

main.py

VisualizeRun

```
1 def validate_segment(length_m):
3
4     if not isinstance(length_m, (int, float)):
5         return False, "Error: Cable length must be a number."
6
7     if length_m <= 0:
8         return False, "Error: Cable length must be greater
          than 0 meters."
9
10    if length_m > 100:
11        return False, "Error: Cable length exceeds the
          maximum limit of 100 meters."
12
13    return True, "Valid: Cable segment satisfies the
          100-meter limit."
14
15
16 # Test cases
17 test_cases = [50, 100, 125]
18
19 for length in test_cases:
20     result, message = validate_segment(length)
21
22     print(f"Cable Length: {length} m")
23     print(f"Result: {result}")
24     print(f"Message: {message}")
25     print("-" * 50)
```

Output

```
ERROR!
Cable Length: 50 m
Result: True
Message: Valid: Cable segment satisfies the 100-meter limit.
-----
Cable Length: 100 m
Result: True
Message: Valid: Cable segment satisfies the 100-meter limit.
-----
Cable Length: 125 m
Result: False
Message: Error: Cable length exceeds the maximum limit of 100
meters.
-----

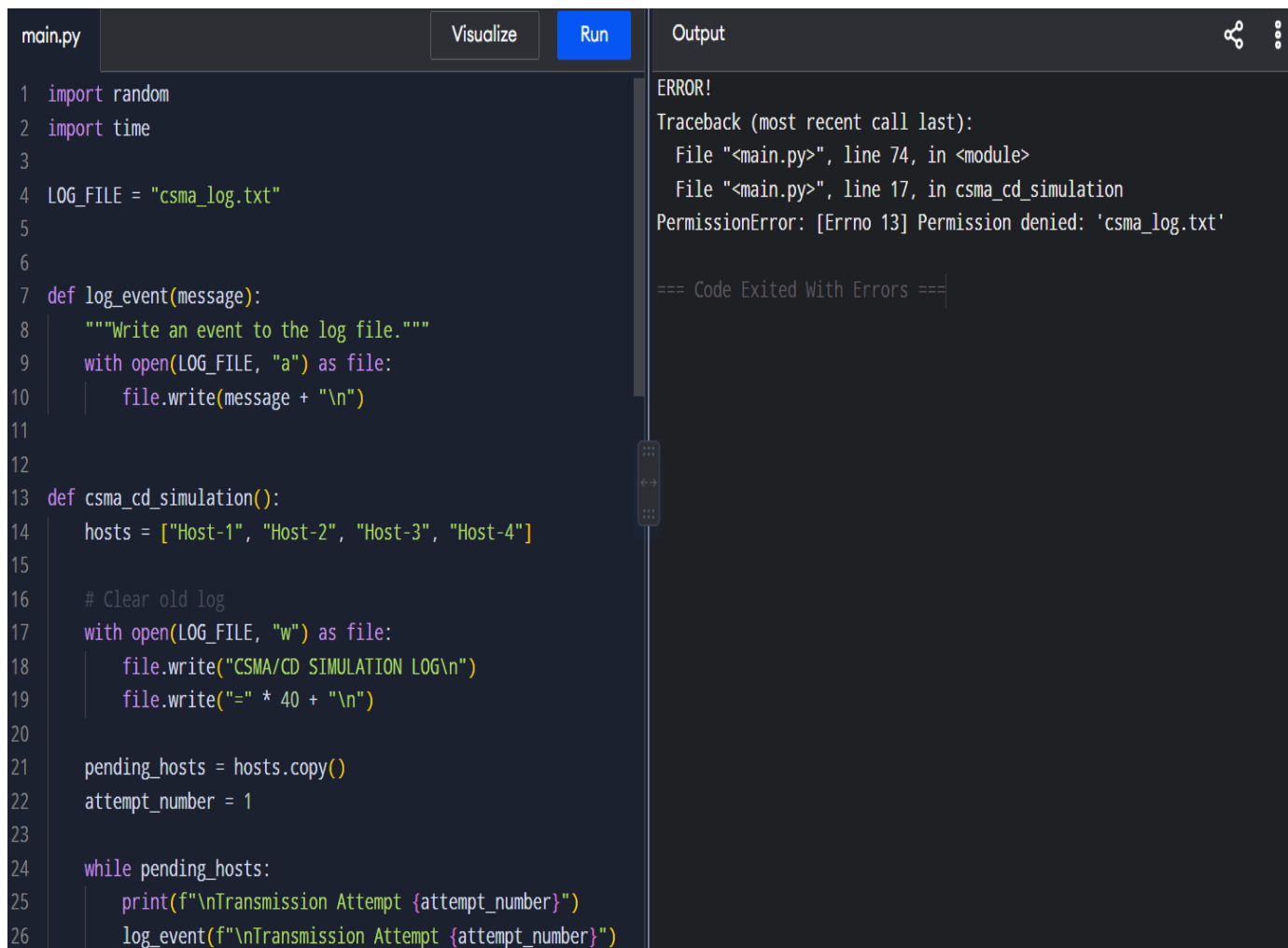
=== Code Execution Successful ===
```

Test Case	Result	Interpretation
50 m	True	Valid cable length
100 m	True	Maximum permissible length
125 m	False	Exceeds permissible limit

3. CSMA/CD Simulation

Approach:

1. Four hosts attempt to transmit.
2. Each host senses the channel.
3. If multiple hosts transmit simultaneously, a collision occurs.
4. A random back-off interval is generated.
5. Hosts retry after waiting.
6. Every event is recorded in csma_log.txt.



The screenshot shows a code editor with a file named `main.py`. The code defines a `log_event` function to write to `csma_log.txt` and a `csma_cd_simulation` function that initializes four hosts, clears the log, and enters a loop for transmission attempts. The output pane shows a `PermissionError` because the program does not have write permissions for `csma_log.txt`.

```
main.py Visualize Run Output
1 import random
2 import time
3
4 LOG_FILE = "csma_log.txt"
5
6
7 def log_event(message):
8     """Write an event to the log file."""
9     with open(LOG_FILE, "a") as file:
10         file.write(message + "\n")
11
12
13 def csma_cd_simulation():
14     hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
15
16     # Clear old log
17     with open(LOG_FILE, "w") as file:
18         file.write("CSMA/CD SIMULATION LOG\n")
19         file.write("=" * 40 + "\n")
20
21     pending_hosts = hosts.copy()
22     attempt_number = 1
23
24     while pending_hosts:
25         print(f"\nTransmission Attempt {attempt_number}")
26         log_event(f"\nTransmission Attempt {attempt_number}")
```

ERROR!
Traceback (most recent call last):
 File "<main.py>", line 74, in <module>
 File "<main.py>", line 17, in csma_cd_simulation
PermissionError: [Errno 13] Permission denied: 'csma_log.txt'

=== Code Exited With Errors ===

Explanation

CSMA/CD means **C**arrier **S**ense **M**ultiple **A**ccess with **C**ollision **D**etection.

- **Carrier Sense:** A host checks whether the medium is busy.
- **Multiple Access:** Multiple hosts share the network.
- **Collision Detection:** Simultaneous transmissions can be detected.
- **Back-off:** After a collision, hosts wait for a randomly selected period before retransmitting.

This reduces repeated collisions and allows hosts to share the Ethernet medium efficiently.

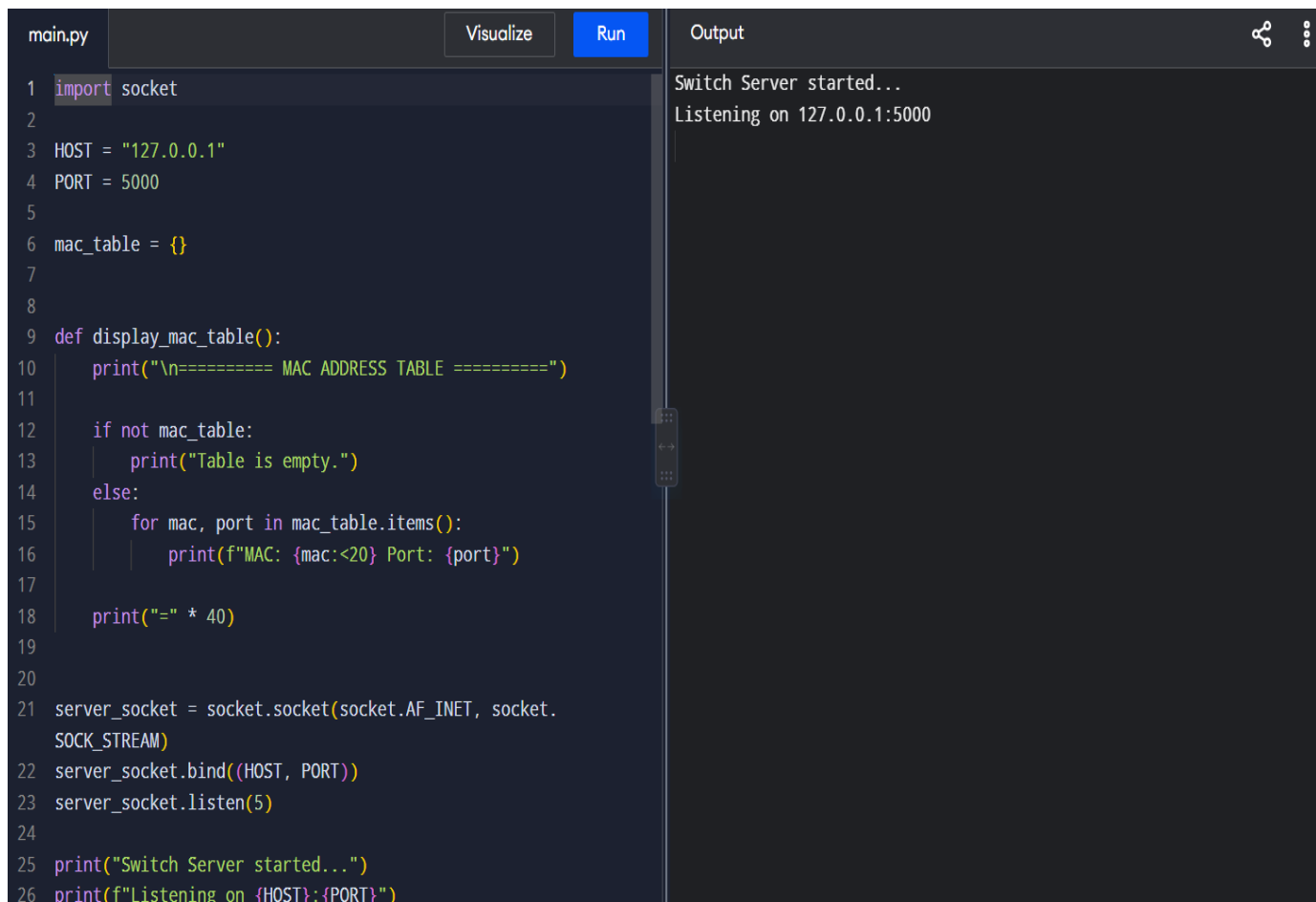
Note: Modern switched full-duplex Ethernet normally does **not** use CSMA/CD. This program is a simulation of the classic shared-medium Ethernet mechanism requested by the question.

4. Client-Server Application with MAC Address Learning

Approach:

The server behaves like a simplified Ethernet switch.

- Client sends its MAC address, destination MAC address and frame.
- Server learns the source MAC address.
- Server stores the MAC address and associated port.
- If destination MAC is known, the frame is forwarded to the appropriate port.
- Otherwise, the frame is flooded to available ports.
- The MAC table is displayed after successful transmission.



```
main.py Visualize Run Output
1 import socket
2
3 HOST = "127.0.0.1"
4 PORT = 5000
5
6 mac_table = {}
7
8
9 def display_mac_table():
10     print("\n===== MAC ADDRESS TABLE =====")
11
12     if not mac_table:
13         print("Table is empty.")
14     else:
15         for mac, port in mac_table.items():
16             print(f"MAC: {mac:<20} Port: {port}")
17
18     print("=" * 40)
19
20
21 server_socket = socket.socket(socket.AF_INET, socket.
22     SOCK_STREAM)
23 server_socket.bind((HOST, PORT))
24 server_socket.listen(5)
25
26 print("Switch Server started...")
27 print(f"Listening on {HOST}:{PORT}")
```

Switch Server started...
Listening on 127.0.0.1:5000

Explanation

An Ethernet switch uses **MAC learning** to build a MAC address table.

When a frame arrives:

1. The switch reads the **source MAC address**.
2. It associates the MAC address with the incoming port.
3. It checks the destination MAC address.
4. If the destination is known, it forwards the frame only through the corresponding port.
5. If the destination is unknown, it floods the frame to other ports.
6. Once the destination responds, the switch can learn its MAC address.

This reduces unnecessary network traffic and improves LAN efficiency.

5. Star Switch Class

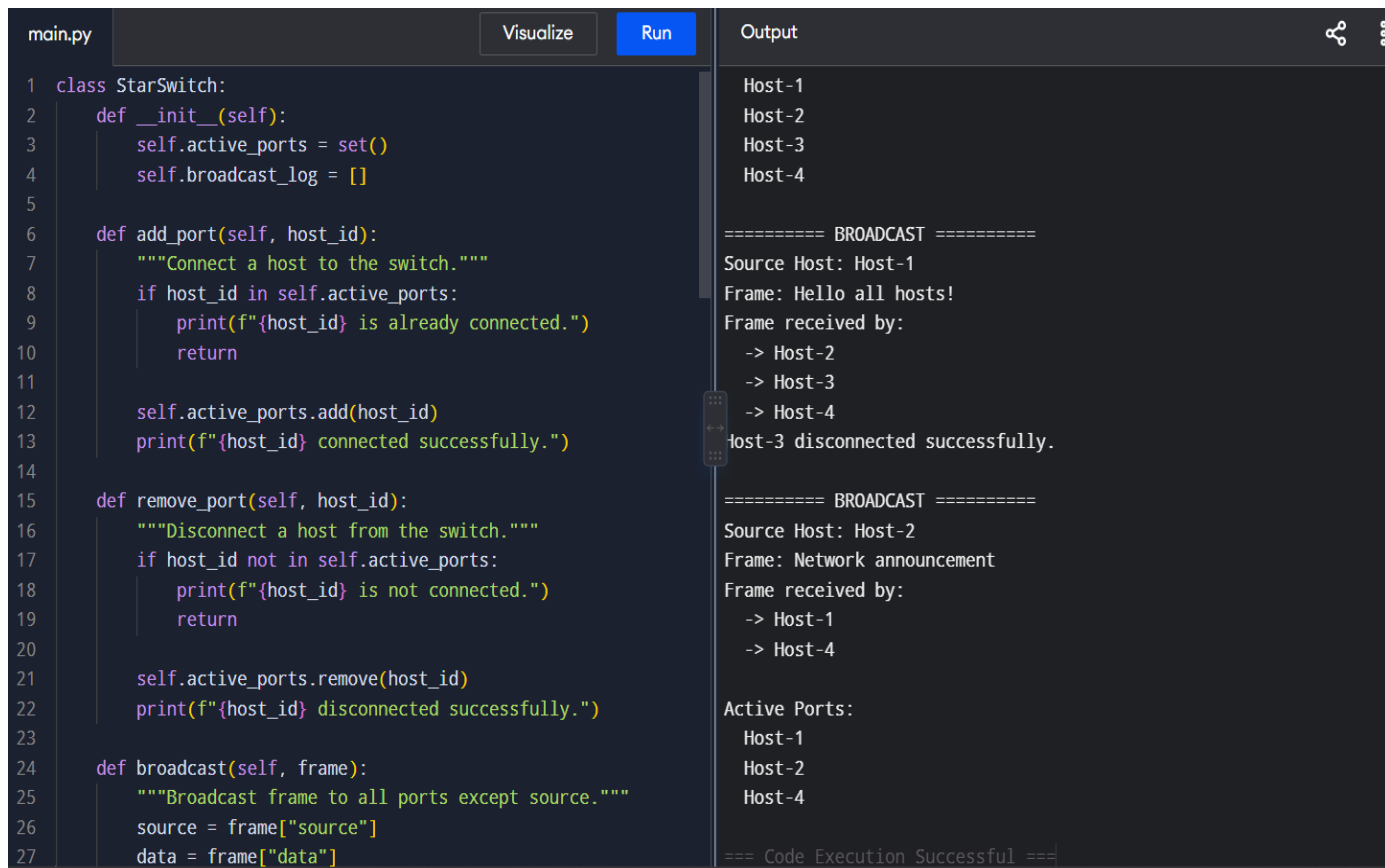
Approach:

The class maintains a collection of active hosts/ports.

Required methods:

- Add port (host_id)
- remove port (host_id)
- broadcast(frame)

Every broadcast operation is logged with the source and receiving hosts.



The screenshot shows a code editor with a file named 'main.py'. The code defines a class 'StarSwitch' with three methods: 'add_port', 'remove_port', and 'broadcast'. The 'add_port' method checks if a host is already connected and adds it to the 'active_ports' set if not. The 'remove_port' method checks if a host is connected and removes it from the 'active_ports' set if so. The 'broadcast' method sends a frame to all active ports except the source. The output window shows the execution of these methods: adding Host-1, Host-2, Host-3, and Host-4; broadcasting a 'Hello all hosts!' frame from Host-1 to Host-2, Host-3, and Host-4; removing Host-3; broadcasting a 'Network announcement' frame from Host-2 to Host-1 and Host-4; and finally listing the active ports as Host-1, Host-2, and Host-4. The execution ends with 'Code Execution Successful'.

```
1 class StarSwitch:
2     def __init__(self):
3         self.active_ports = set()
4         self.broadcast_log = []
5
6     def add_port(self, host_id):
7         """Connect a host to the switch."""
8         if host_id in self.active_ports:
9             print(f"{host_id} is already connected.")
10            return
11
12            self.active_ports.add(host_id)
13            print(f"{host_id} connected successfully.")
14
15    def remove_port(self, host_id):
16        """Disconnect a host from the switch."""
17        if host_id not in self.active_ports:
18            print(f"{host_id} is not connected.")
19            return
20
21            self.active_ports.remove(host_id)
22            print(f"{host_id} disconnected successfully.")
23
24    def broadcast(self, frame):
25        """Broadcast frame to all ports except source."""
26        source = frame["source"]
27        data = frame["data"]
```

Output:

```
Host-1
Host-2
Host-3
Host-4

===== BROADCAST =====
Source Host: Host-1
Frame: Hello all hosts!
Frame received by:
-> Host-2
-> Host-3
-> Host-4
Host-3 disconnected successfully.

===== BROADCAST =====
Source Host: Host-2
Frame: Network announcement
Frame received by:
-> Host-1
-> Host-4

Active Ports:
Host-1
Host-2
Host-4

=== Code Execution Successful ===
```

Explanation

Broadcasting allows a frame to reach **all active hosts except the source host**. It is useful when the sender does not know the destination MAC address or when a network protocol requires communication with multiple devices.

In a star topology, the central switch controls the communication between connected hosts. Therefore, the switch provides centralized management and makes adding or removing hosts relatively simple.

